

اپیزود اول: پایه‌گذاری سیستم و طرح مسئله (The Foundation)

در این اپیزود، بررسی می‌کنیم که اصلاً درد اصلی چه بود و ما در پایین‌ترین لایه پروژه چه چیزی ساختیم.

۱. اصلاً چرا این پروژه؟ (What & Why)

- **مشکل (The Problem):** ماشین‌های امروزی پر از سنسور و دوربین هستند و وظایف پردازشی سنگینی (مثل مسیریابی هوشمند، تحلیل ویدیو و تصمیم‌گیری) تولید می‌کنند. اما کامپیوتر داخل ماشین ضعیف است.
- **راه‌حل اولیه (VEC):** ایده این است که ماشین‌ها این وظایف سنگین را از طریق اینترنت بی‌سیم به آنتن‌های پردازشی کنار جاده (RSU) بفرستند تا آن‌ها پردازش کنند. به این کار می‌گویند محاسبات لبه خودرویی (Vehicular Edge Computing).
- **چالش اصلی (Mobility):** ماشین‌ها ثابت نیستند! ماشین با سرعت در حال حرکت است. اگر وظیفه را به یک RSU بفرستد و قبل از تمام شدن پردازش، از محدوده آنتن‌دهی آن RSU خارج شود، چه اتفاقی می‌افتد؟ کار نیمه‌تمام می‌ماند و سیستم مختل می‌شود. ما به سیستمی نیاز داشتیم که این حرکت را پیش‌بینی کند و وظایف را هوشمندانه پخش کند.

۲. ما چه کار کردیم؟ (The Implementation)

- **ساخت دنیای واقعی (SUMO):** ما نمی‌خواستیم الگوریتم را روی یک محیط فرضی پایتونی با اعداد تصادفی تست کنیم. ما با استفاده از SUMO یک نقشه شهری کاملاً واقعی با تقاطع‌ها، خیابان‌ها و ۵۰۹ خودروی در حال حرکت ساختیم تا همه‌چیز دقیقاً مطابق فیزیک دنیای واقعی باشد.
- **ایجاد چشم‌های سیستم (TraCI):** پایتون به طور پیش‌فرض نمی‌تواند داخل نرم‌افزار SUMO را ببیند. ما از کتابخانه TraCI استفاده کردیم تا یک "پل ارتباطی" بین پایتون و SUMO بسازیم.
- **کاشت زیرساخت (RSUs):** ما ۶ دستگاه RSU را در مختصات استراتژیک نقشه (تقاطع‌ها) قرار دادیم.
- **حلقه کنترلی (The Step):** با استفاده از تابع `step()`، پایتون ما در هر ثانیه شبیه‌سازی، زمان را متوقف می‌کند، مختصات دقیق (X, Y) تکتک ماشین‌ها را می‌خواند، محاسبه می‌کند که هر ماشین در شعاع کدام RSU است، و سپس به شبیه‌ساز اجازه می‌دهد یک ثانیه دیگر جلو برود.

خلاصه اپیزود ۱: ما در این فاز، زمین بازی را ساختیم. ما یک شهر زنده و متحرک داریم که پایتون مثل یک کنترل‌کننده همه‌کاره، روی تکتک فریم‌های آن نظارت کامل دارد.

اپیزود دوم: تولید وظایف و مدل‌سازی فیزیکی مسئله (Tasks & Mathematics)

در اپیزود اول ما زمین بازی (خیابان‌ها و ماشین‌ها) را ساختیم. اما در آن مرحله، ماشین‌ها صرفاً جعبه‌هایی بودند که در خیابان حرکت می‌کردند. در این اپیزود بررسی می‌کنیم که چطور این شبیه‌سازی ترافیکی را به یک «شبیه‌سازی شبکه‌های کامپیوتری و پردازشی» تبدیل کردیم.

۱. اصلاً چرا به این بخش نیاز داشتیم؟ (The Why)

- **معنا دادن به پردازش لبه:** در دنیای واقعی، ماشین خودران در هر ثانیه در حال تولید گیگابایت‌ها داده (از طریق رادارها و دوربین‌ها) است که باید فوراً تحلیل شوند. ما باید این مفهوم را در کدهایمان شبیه‌سازی می‌کردیم تا سیستم چیزی برای "پردازش کردن" داشته باشد.
- **متریک‌های اندازه‌گیری (Metrics):** از کجا باید می‌فهمیدیم که سیستم ما خوب کار می‌کند یا بد؟ در مهندسی کامپیوتر، "خوب بودن" باید قابل اندازه‌گیری باشد. ما به معیارهای دقیقی مثل تاخیر (Delay) و مصرف انرژی (Energy) نیاز داشتیم.

- **سیگنال یادگیری برای هوش مصنوعی:** شبکه‌های یادگیری تقویتی (مثل شبکه ما) کور هستند؛ آن‌ها فقط با گرفتن "پاداش" یا "جریمه" یاد می‌گیرند. ما باید قوانین فیزیک و شبکه‌های مخابراتی را به زبان ریاضی و پاداش ترجمه می‌کردیم تا شبکه بتواند آن‌ها را درک کند.

۲. ما دقیقاً چه کار کردیم؟ (The Implementation)

- **کارخانه تولید وظیفه (Task Generation):** ما اسکریپتی (`task_generator.py`) نوشتیم که در هر لحظه به صورت تصادفی اما منطقی، برای ماشین‌ها یک "وظیفه پردازشی" (Task) تولید می‌کند. این وظیفه دو ویژگی حیاتی دارد: حجم داده برای پردازش (چقدر سنگین است) و مهلت مجاز یا Deadline (چقدر زمان داریم تا نتیجه را برگردانیم).
- **مدل‌سازی تاخیر و انرژی (Math Models):** در فایل `models.py`، فرمول‌های پیچیده ریاضی مقاله را به کدهای پایتون تبدیل کردیم. ما محاسبه کردیم که اگر یک وظیفه به RSU شماره یک برود، با توجه به قدرت پردازنده‌اش، دقیقاً چند میلی‌ثانیه طول می‌کشد تا حل شود و چند ژول انرژی مصرف می‌کند.
- **تعریف خط قرمز سیستم (Reward Function):** ما هدف نهایی کل پروژه را در یک فرمول پاداش خلاصه کردیم. فرمول ما می‌گوید: «پاداش برابر است با منفی مجموع تاخیر و انرژی». یعنی هرچه تاخیر و انرژی کمتر باشد، پاداش بیشتر است. اما یک شرط اساسی هم گذاشتیم: اگر مجموع تاخیر پردازش از Deadline وظیفه بیشتر شد، یک جریمه بسیار سنگین (۱۰۰-) به سیستم اعمال کن.

خلاصه اپیزود ۲: در این فاز، ما قوانین فیزیک، پردازش و محدودیت‌های زمانی را به شبیه‌ساز تزریق کردیم. حالا ما سیستمی داریم که در آن ماشین‌ها وظایف پردازشی سنگینی تولید می‌کنند که هر میلی‌ثانیه تاخیر در انجام آن‌ها، هزینه و جریمه در پی دارد.

اما هنوز یک مشکل بزرگ پابرجاست: ما می‌دانیم هزینه پردازش چقدر است، اما اگر ماشین قبل از اتمام پردازش با سرعت از زیر آنتن دور شد، چه خاکی بر سر بریزیم؟

اپیزود سوم: مدل‌سازی تحرک و تله‌پاتی ماشین‌ها با گراف (GAT+GRU)

در دو اپیزود قبلی، ما نقشه را ساختیم و به ماشین‌ها وظیفه دادیم. اما یک چالش مهلک در محیط‌های Vehicular (خودرویی) وجود دارد که روش‌های سنتی را فلج می‌کند: «حرکت».

۱. اصلاً چرا به این بخش نیاز داشتیم؟ (The Why)

- **خطر خروج از پوشش:** فرض کنید یک ماشین وظیفه پردازشی سنگینی را به RSU شماره ۱ می‌فرستد. RSU شروع به پردازش می‌کند و می‌بیند ۳ ثانیه طول می‌کشد. اما ماشین با سرعت ۸۰ کیلومتر بر ساعت در حال حرکت است و تنها ۲ ثانیه دیگر در محدوده آنتن‌دهی این RSU قرار دارد. چه اتفاقی می‌افتد؟ پردازش تمام می‌شود، اما ماشین دیگر آنجا نیست تا جواب را بگیرد! کل انرژی و زمان هدر رفت.
- **نیاز به پیش‌بینی T_{stay} :** ما باید قبل از تصمیم‌گیری می‌دانستیم که هر ماشین دقیقاً چقدر دیگر در شعاع پوشش این آنتن باقی می‌ماند (زمانی که به آن T_{stay} می‌گوییم).
- **چرا گراف؟** روش‌های سنتی فقط سرعت فعلی ماشین را در نظر می‌گرفتند. اما در دنیای واقعی، اگر ماشین جلویی ترمز کند، ماشین شما هم ترمز می‌کند. ماشین‌ها روی حرکت یکدیگر تأثیر می‌گذارند. ما به شبکه‌ای نیاز داشتیم که این «تأثیرات متقابل» را درک کند.

۲. ما دقیقاً چه کار کردیم؟ (The Implementation)

- **تبدیل ترافیک به گراف (Graph Building):** در فایل `vec_env.py`، کدی نوشتیم که در هر ثانیه، فاصله تمام ماشین‌ها را از هم حساب می‌کند. اگر فاصله دو ماشین کمتر از یک حد مشخص (مثلاً ۱۰۰ متر) بود، بین آن‌ها یک خط

(Edge) کشیدیم. با این کار، خیابان‌های ما تبدیل به یک «گراف پویا» شد که گره‌هایش (Nodes) ماشین‌ها هستند و یال‌هایش (Edges) نشان‌دهنده همسایگی آن‌هاست.

- **استخراج ویژگی‌های مکانی با GAT:** ما از شبکه‌های عصبی گراف (Graph Attention Network) استفاده کردیم. این شبکه به ماشین‌ها اجازه می‌دهد به همسایه‌های خود «توجه» کنند. یعنی شبکه می‌فهمد که محیط اطراف ماشین خلوت است یا ترافیک سنگین است.
- **پیش‌بینی زمان با GRU:** حالا که وضعیت مکانی مشخص شد، باید زمان (آینده) را پیش‌بینی می‌کردیم. ما از شبکه بازگشتی GRU (به شکل Encoder-Decoder) استفاده کردیم. این شبکه مسیر حرکت قبلی ماشین را می‌بیند، وضعیت گراف همسایه‌ها را بررسی می‌کند و مسیر حرکت ماشین در ثانیه‌های آینده را پیش‌بینی می‌کند.
- **محاسبه هوشمندانه:** با داشتن مسیر آینده، سیستم ما حالا با قطعیت بالا تخمین می‌زند که این ماشین مثلاً دقیقاً ۱۵ ثانیه دیگر در پوشش این آنتن باقی می‌ماند.

خلاصه اپیزود ۳: در این فاز، ما به سیستم قدرت پیش‌گویی (Foresight) دادیم. هوش مصنوعی ما حالا فقط یک ماشین منزوی را نمی‌بیند، بلکه کل ترافیک را مثل یک شبکه به هم پیوسته (گراف) تحلیل می‌کند و می‌داند هر ماشین چه زمانی از شبکه خارج می‌شود.

اپیزود چهارم: مدیریت بحران و الگوریتم اولویت‌بندی صف (Task Prioritization)

در اپیزودهای قبل وظایف تولید شدند و زمان خروج ماشین‌ها (T_{stay}) هم پیش‌بینی شد. اما وقتی چند ماشین همزمان وظایفشان را به یک RSU می‌فرستند، صفی از کارهای نکرده تشکیل می‌شود.

۱. اصلاً چرا به این بخش نیاز داشتیم؟ (The Why)

- **فاجعه صف‌های سنتی (FIFO):** در سیستم‌های عادی، هر وظیفه‌ای که زودتر برسد، زودتر پردازش می‌شود (First-In-First-Out). اما فرض کنید ماشین "الف" پشت چراغ قرمز ایستاده و یک وظیفه سبک دارد که ۵ ثانیه مهلت دارد. ماشین "ب" با سرعت بالا در حال عبور است و یک وظیفه حیاتی دارد که فقط ۱ ثانیه مهلت دارد. اگر ماشین "الف" زودتر درخواست داده باشد، در سیستم سنتی کار ماشین "ب" در صف می‌ماند، مهلتش تمام می‌شود و با شکست مواجه می‌گردد.
- **محدودیت منابع RSU:** آنتن‌های لبه (Edge) برعکس سرورهای ابری (Cloud) پردازنده‌های محدودی دارند و نمی‌توانند همه کارها را همزمان در کسری از ثانیه انجام دهند. پس ما به یک "مدیر بحران" نیاز داشتیم تا بگوید کدام کار الان واجب‌تر است.

۲. ما دقیقاً چه کار کردیم؟ (The Implementation)

- **تبدیل منطق به فرمول ریاضی:** ما فرمول اولویت‌بندی (معادله ۲۳ مقاله) را در فایل `models/priority.py` کدنویسی کردیم. این فرمول برای هر وظیفه یک "نمره اولویت" (Priority Score) محاسبه می‌کند.
- **معیارهای سه‌گانه امتیازدهی:** این فرمول هوشمند، نمره را بر اساس سه پارامتر اساسی می‌دهد:
 ۱. **زمان ماندگاری (T_{stay}):** که از مدل گراف (GAT+GRU) گرفتیم. هرچه ماشین زودتر از پوشش خارج شود، کارش اورژانسی‌تر است و نمره بالاتری می‌گیرد.
 ۲. **مهلت مجاز (d):** وظایفی که ددلاین (Deadline) سفت‌وسخت‌تری دارند، اولویت بیشتری می‌گیرند.
 ۳. **حجم وظیفه (ρ):** میزان منابعی که کار برای پردازش نیاز دارد.
- **مرتب‌سازی لحظه‌ای (Dynamic Sorting):** در تابع `step` در فایل `vec_env.py`، کدی نوشتیم که در هر ثانیه از شبیه‌سازی، تمام وظایف داخل صفِ هر RSU را بر اساس این نمره اولویت، از بیشترین به کمترین مرتب (Sort) می‌کند.

خلاصه اپیزود ۴: در این فاز، ما صف‌های احمقانه (که فقط به نوبت نگاه می‌کردند) را به صف‌های هوشمند و پویا تبدیل کردیم. حالا سیستم ما می‌فهمد که وظیفه یک ماشین تندرو با ددلاین کوتاه، بسیار مهم‌تر از وظیفه یک ماشین پارک‌شده است و جلوی

هدر رفتن وظایف را می‌گیرد.

حالا همه‌چیز برای تصمیم‌گیری نهایی آماده است. زمین بازی، وظایف، پیش‌بینی آینده و صف‌های هوشمند. تنها چیزی که می‌ماند، رهبر ارکستر است که باید تصمیم بگیرد وظایف بین RSU ها چطور پخش شوند.

اپیزود پنجم: تولد مغز متفکر (یادگیری تقویتی A3C) و رقبای سنتی

در چهار اپیزود قبلی، ما فونداسیون را چیدیم: ماشین‌ها حرکت می‌کنند، گراف شکل می‌گیرد، آینده پیش‌بینی می‌شود و صف‌ها هوشمندانه مرتب می‌شوند. اما هنوز یک سوال اساسی بی‌پاسخ مانده است: "این وظیفه در نهایت باید به کدام RSU فرستاده شود؟"

۱. اصلاً چرا به این بخش نیاز داشتیم؟ (The Why)

- **نیاز به همکاری (RSU Collaboration):** گاهی اوقات RSU فعلی که ماشین زیر آن قرار دارد، به شدت شلوغ است. اگر ماشین وظیفه‌اش را به آنجا بفرستد، قطعاً مهلت مجاز (Deadline) تمام می‌شود. ما سیستمی می‌خواستیم که بتواند وظیفه را به RSU های همسایه منتقل کند تا آن‌ها به صورت مشارکتی کار را انجام دهند.
- **شکست قوانین ساده:** حل این مسئله با دستورات ساده‌ی `if-else` غیرممکن است. اگر همیشه وظیفه را به خلوت‌ترین RSU بفرستیم (روش Greedy)، ممکن است ماشین اصلاً به سمت آن RSU در حرکت نباشد و ارتباط قطع شود. ما به یک هوش مصنوعی نیاز داشتیم که «عواقب بلندمدت» تصمیماتش را درک کند.

۲. ما دقیقاً چه کار کردیم؟ (The Implementation)

- **تعریف حواس برای هوش مصنوعی (State):** شبکه عصبی کور است. ما در فایل `vec_env.py` کدی نوشتیم که در هر لحظه، وضعیت کل سیستم را به یک آرایه عددی (تانسور) تبدیل کند. این آرایه شامل مختصات ماشین، حجم وظیفه، مهلت مجاز و میزان شلوغی تمام ۶ دستگاه RSU است. این آرایه همان چیزی است که هوش مصنوعی ما می‌بیند.
- **پیاده‌سازی الگوریتم A3C:** ما در فایل `a3c_agent.py`، پیشرفته‌ترین عامل یادگیری تقویتی یعنی Actor-Critic را کدنویسی کردیم. این معماری دو بخش دارد:

۱. **بازیگر (Actor):** با دیدن وضعیت، تصمیم می‌گیرد وظیفه به کدام یک از ۶ آنتن فرستاده شود.

۲. **منتقد (Critic):** تصمیم بازیگر را بررسی می‌کند و به آن نمره می‌دهد تا در دفعات بعدی بهتر عمل کند.

- **حلقه آموزش (Training Loop):** در فایل `train.py`، ما سیستم را برای ۵۰ اپیزود رها کردیم تا در شبیه‌ساز بچرخد و خطا کند. هر بار که تصمیم احمقانه‌ای می‌گرفت (تاخیر زیاد می‌شد)، فرمول پاداش به شدت او را جریمه می‌کرد و هر بار که با همکاری RSU ها مصرف انرژی را کاهش می‌داد، پاداش می‌گرفت. شبکه با این روش وزن‌های خود را آپدیت کرد و یاد گرفت.
- **ساخت رقبای ضعیف (Baselines):** در مقالات علمی، برای اثبات هوش سیستم، باید آن را با افراد خنگ مقایسه کنیم! ما دو فایل `greedy_agent.py` و `local_agent.py` را ساختیم. روش `Local` اصلاً اهل همکاری نبود و همیشه کارها را در همان RSU اول پردازش می‌کرد (که باعث تاخیر می‌شد). روش `Greedy` هم مثل انسان‌های عجول، فقط به خلوت بودن RSU در همان لحظه نگاه می‌کرد و آینده را نمی‌دید.

خلاصه اپیزود ۵ (و پایان): در این فاز نهایی، ما «رهبر ارکستر» را خلق کردیم. هوش مصنوعی ما (CoTOP) با نگاه کردن به گراف حرکت ماشین‌ها و شلوغی آنتن‌ها، بهترین تصمیم ممکن را می‌گیرد، رقبا را شکست می‌دهد و کمترین تاخیر و مصرف انرژی را در کل سیستم ثبت می‌کند.

اپیزود ۶: گذر از دمو به واقعیت (The Reality Check)

در ۵ اپیزود اول، ما فونداسیون سیستم را ساختیم، اما یک مشکل بزرگ داشتیم: کدهای ارزیابی ما فقط یک "ماکت" بودند. ما برای روز دفاع به یک آزمایشگاه واقعی نیاز داشتیم.

۱. اصلاً چرا به این بخش نیاز داشتیم؟ (The Why)

- ما فهمیدیم که رقبای ما (Greedy و DDQN) باگ دارند و در شرایط ناعادلانه‌ای مقایسه می‌شوند.
- محیط شبیه‌ساز ما (vec_env.py) هنوز قابلیت‌های اصلی مقاله یعنی "پردازش مشارکتی بین دو آنتن" و "تولید همزمان وظایف" را نداشت.
- اسکرپت ablation.py فقط رشته‌های متنی با اعداد ساختگی (Hardcoded) چاپ می‌کرد و هیچ پشتوانه علمی‌ای نداشت.

۲. ما دقیقاً چه کار کردیم؟ (The Implementation)

- **جراحی قلب سیستم (vec_env.py):** ما محیط شبیه‌ساز را بازنویسی کردیم. ابتدا کلیدهای خاموش/روشن (use_mobility_detector ، use_priority ، use_collaboration) را به آن اضافه کردیم. سپس در تابع step ، منطق مقاله را پیاده کردیم. اگر زمان خروج ماشین (T_{stay}) کمتر از زمان پردازش بود، سیستم به جای رها کردن وظیفه، آن را بین دو RSU تقسیم می‌کند (Collaboration).
- **احیای رقبای:** فایل greedy_agent.py را اصلاح کردیم تا واقعاً طول صف‌ها را بشمارد. همچنین فایل ddqn_agent.py که قبلاً آموزش نمی‌دید را با اضافه کردن توابع loss.backward () و optimizer.step () به یک رقیب سرسخت و واقعی تبدیل کردیم.
- **ماشین حقیقت‌سنج (CSV Logging):** ما تابع log_episode را به تمام فایل‌های آموزش و تست اضافه کردیم. از این به بعد، سیستم به جای چاپ کردن اعداد الکی، نتایج هر اپیزود را در فایل‌های واقعی (مثل cotop_log.csv) ذخیره می‌کرد.
- **رسم نمودار خودکار (evaluate.py):** فایلی نوشتیم که به طور خودکار این CSV ها را می‌خواند و نمودارهای میله‌ای مقایسه‌ای (مانند تصاویری که فرستادی) را رسم می‌کند.

خلاصه اپیزود ۶: در این فاز، ما پروژه را از یک "کاردستی دانشجویی" به یک "پایپ‌لاین تحقیقاتی استاندارد" تبدیل کردیم. حالا هر عددی که تولید می‌شود، پشتوانه واقعی و دیتاست ثبت‌شده دارد.

اپیزود ۷: عملیات نجات مغز سیستم و معمای Advantage

در اپیزود قبلی ما سعی کردیم با فایل train_robust.py به سیستم شجاعت بدهیم تا همه آنتن‌ها را تست کند (اکتشاف یا Exploration). اما با یک باگ ریاضی سیستم را به کما بردیم.

۱. مشکل ریاضی دقیقاً چه بود؟ (The Problem)

در یادگیری تقویتی، فرمول پایه برای آموزش بازیگر (Actor) این است:

$$Loss = -\log(p) \times Reward$$

اگر پاداش شما مثبت باشد (مثلاً ۱۰ امتیاز)، این فرمول به درستی کار می‌کند و شبکه تشویق می‌شود. اما در محیط Vehicular Edge Computing ما، پاداش‌ها نشان‌دهنده "هزینه و تاخیر" هستند و همیشه اعدادی به شدت منفی‌اند (مثلاً -۲۰۰۰۰).

وقتی شبکه این عدد منفی بزرگ را در فرمول می‌گذارد، Loss به یک عدد مثبت و بسیار بزرگ تبدیل می‌شود. شبکه عصبی برای کم کردن این Loss، سعی می‌کند احتمال (p) را به صفر برساند. در نتیجه، شبکه به جای یادگیری، کور می‌شود و فقط یک خروجی ثابت یا نویز تولید می‌کند. به همین دلیل بود که در تست Ablation خاموش کردن ماژول‌ها هیچ تأثیری نداشت؛ سیستم اصلاً به داده‌های محیط (State) نگاه نمی‌کرد!

۲. ما چه کار کردیم؟ (The Solution: Advantage Actor-Critic)

برای حل این مشکل، ما مفهوم «مزیت» (Advantage) را به فایل `train_robust.py` اضافه می‌کنیم.

ایده این است: ما نباید به شبکه بگوییم "پاداش تو -۲۰۰۰۰ بود، پس کارت افتضاح بود!"، چون شاید اگر کار دیگری می‌کرد پاداشش -۳۰۰۰۰ می‌شد! ما باید عملکرد شبکه را با "انتظارات خودش" مقایسه کنیم.

ما از بخش Critic (منتقد) در شبکه عصبی می‌پرسیم: "انتظار داشتی در این وضعیت چقدر پاداش بگیری؟" (این همان متغیر `state_value` است).

سپس فرمول Advantage را این‌گونه تعریف می‌کنیم:

$$Advantage = Reward - state_value$$

حالا اگر پاداش واقعی از چیزی که منتقد انتظار داشت بهتر (یعنی کمتر منفی) باشد، مقدار Advantage مثبت می‌شود و شبکه تشویق می‌گردد.

۳. پیاده‌سازی در کد (`train_robust.py`)

ما فایل `train_robust.py` را باز می‌کنیم و بخش محاسبه Loss (درون حلقه آموزش) را به این شکل ارتقا می‌دهیم تا این ریاضیات درست کار کند:

Python

```
# train_robust.py در فایل step کدهای داخل حلقه

action_probs, state_value = agent(state_tensor)

# تزریق شجاعت (Epsilon-Greedy)
if random.random() < epsilon:
    action = random.randint(0, 5)
else:
    action = torch.argmax(action_probs).item()

next_state, reward, done, _ = env.step(action)
total_reward += reward

# --- Advantage محاسبه --- بخش جدید و حیاتی ریاضیات ---
reward_tensor = torch.tensor([reward], dtype=torch.float32)

# پاداش واقعی منهای پیش‌بینی منتقد Advantage محاسبه
advantage = reward_tensor - state_value.detach()

# Advantage با استفاده از Actor (بازیگر Loss محاسبه)
actor_loss = -torch.log(action_probs[action] + 1e-10) *
advantage

# با خطای میانگین مربعات (Critic) منتقد Loss محاسبه
critic_loss = torch.nn.functional.mse_loss(state_value,
reward_tensor)

# مجموع خطاها و آپدیت وزن‌ها
```

```
total_loss = actor_loss + critic_loss
```

```
optimizer.zero_grad()
```

```
total_loss.backward()
```

```
optimizer.step()
```

```
# -----
```

خلاصه اپیزود ۷: با این تغییر کوچک اما حیاتی، ما سیستم را از شر اعداد منفی مطلق خلاص کردیم. حالا شبکه A3C می‌فهمد که حتی اگر پاداشش منفی است، مهم این است که نسبت به وضعیت‌های دیگر عملکرد **بهتری** داشته باشد. این تغییر باعث می‌شود سیستم واقعاً یاد بگیرد، ستون CoTOP در نمودارها به بالاترین حد خود برسد و تست Ablation به درستی افت عملکرد ماژول‌ها را در صورت خاموش شدن نشان دهد.

اپیزود ۸: کالبدشکافی شکست (چرا همه‌چیز در هوش مصنوعی اشتباه پیش می‌رود؟)

وقتی در اپیزودهای قبلی نمودارهای ما کاملاً مسطح شدند یا سیستم ما از یک الگوریتم ساده (Greedy) شکست خورد، این یک باگ برنامه‌نویسی نبود؛ بلکه یک شکست رفتاری در هوش مصنوعی بود. اما چرا این اتفاقات رخ می‌دهند؟

۱. سندرم حیوان وحشت‌زده (تله پاداش‌های منفی)

- **مشکل:** یادگیری تقویتی بر اساس سیستم پاداش و تنبیه کار می‌کند. در پروژه ما، پاداش‌ها نشان‌دهنده "تاخیر و مصرف انرژی" بودند که همیشه اعدادی منفی و بزرگ هستند (مثل ۱۰۰- برای از دست دادن مهلت مجاز).
- **دلیل شکست:** وقتی یک شبکه عصبی در همان ابتدای کار فقط با جریمه‌های سنگین و اعداد منفی بزرگ بمباران می‌شود، دقیقاً مثل یک حیوان وحشت‌زده عمل می‌کند. او به جای اینکه سعی کند راه‌های جدیدی برای موفقیت پیدا کند، "یخ می‌زند" (Freeze). شبکه به این نتیجه می‌رسد که: «هر کاری می‌کنم جریمه می‌شوم، پس بهتر است همیشه یک کار تکراری بی‌خطر (مثل فرستادن وظیفه به همان RSU اول) را انجام دهم تا حداقل جریمه‌ام قابل پیش‌بینی باشد!» این دقیقاً همان پدیده‌ای است که به آن **Policy Collapse** می‌گوییم.

۲. گیر افتادن در منطقه امن (Local Optima)

- **مشکل:** هوش مصنوعی ما در نسخه‌های اولیه حاضر نبود وظایف را به آنتن‌های دیگر بفرستد و دقیقاً مثل عامل Local عمل می‌کرد.
- **دلیل شکست:** شبکه عصبی در ابتدای کار تصادفی عمل می‌کند. فرض کنید در اپیزود اول، یک بار وظیفه را به RSU شماره ۳ می‌فرستد، اما چون هنوز زمان خروج را درست پیش‌بینی نمی‌کند، ماشین از محدوده خارج شده و جریمه سنگینی می‌گیرد. از آن به بعد، شبکه از RSU شماره ۳ می‌ترسد! او در یک "بهینه محلی" (Local Optima) گیر می‌افتد؛ یعنی یک رامحل بد اما امن پیدا می‌کند و دیگر حاضر نیست برای پیدا کردن "بهترین رامحل" ریسک کند. به همین دلیل ما مجبور شدیم مکانیزم شجاعت یا **Epsilon-Greedy** را به آن تزریق کنیم تا به زور آنتن‌های دیگر را تست کند.

۳. نابینایی در برابر اعداد (تداخل مقیاس‌ها)

- **مشکل:** در کدهای اولیه، شبکه ما اصلاً متوجه نمی‌شد که صف یک RSU شلوغ است یا مهلت مجاز یک وظیفه در حال اتمام است.
- **دلیل شکست:** شبکه‌های عصبی فقط "اعداد" را می‌بینند. در سیستم ما، مختصات ماشین روی نقشه اعدادی مثل ۱۵۰۰ یا ۲۰۰۰ بودند، اما حجم پردازشی وظیفه عددی بین ۲ تا ۵ بود. وقتی این اعداد به شبکه داده می‌شوند، وزن‌های شبکه کاملاً تحت سلطه اعداد بزرگ (مختصات) قرار می‌گیرند و متغیرهای حیاتی کوچک (مثل حجم کار) عملاً در نویز گم می‌شوند. این دلیل همان جراحی مهمی بود که برای نرمال‌سازی (Normalization) در `vec_env.py` انجام دادیم تا همه اعداد بین ۰ تا ۱ قرار بگیرند و شبکه دیگر کور نباشد.

۴. زنجیره وابستگی‌های پنهان (تست Ablation مسطح)

- **مشکل:** خاموش کردن ماژول گراف، هیچ تاثیری در خروجی سیستم نداشت.
- **دلیل شکست:** یک سیستم هوشمند مثل زنجیر است. وقتی حلقه آخر زنجیر (یعنی شبکه Actor که باید اکشن را انتخاب کند) به خاطر مشکل Advantage و پاداش‌های منفی کاملاً از کار افتاده باشد و فقط یک خروجی ثابت بدهد، دیگر فرقی نمی‌کند که حلقه‌های قبلی زنجیر (مثل پیش‌بینی زمان خروج با گراف) چقدر عالی کار کنند. شبکه اصلاً به آن داده‌ها نگاه نمی‌کند! حل کردن مشکل آخر زنجیر، به طور ناگهانی ارزش تمام کارهای قبلی را نمایان کرد.

خلاصه اپیزود ۸: شکست خوردن در RL به معنای بد بودن کد شما نیست؛ بلکه نشان‌دهنده این است که شما در حال آموزش یک "موجود خنگ اما به شدت منطقی" هستید که همیشه کوتاه‌ترین و احمقانه‌ترین راه را برای فرار از جریمه پیدا می‌کند. وظیفه ما به عنوان مهندس این است که با ریاضیات (مثل Advantage) و منطق (مثل Exploration)، راه فرار را ببندیم و او را مجبور به یادگیری کنیم.